

GTM & SEMI-TECHNICAL PLAYBOOK

Connector OS

Problem, Solution, Use Cases, Plugins & Motion

Everything a GTM lead, founder, or semi-technical buyer champion needs to explain Connector without reading the architecture bible.

GlobalSushrut, Inc.

May 27, 2026

Companion docs: [CONNECTOR_SYSTEM_ARCHITECTURE.pdf](#) (technical depth), [CONNECTOR_COFOUNDER_CLARITY_2026.pdf](#) (pilot & wedge).

Contents

1	Executive Summary	1
1.1	What Connector is (one sentence)	1
1.2	What we are not	1
1.3	The three pillars buyers remember	1
1.4	Product shape: one node, many workloads	1
1.5	Entry wedge (how we sell in 2026)	2
1.6	Stage	2
1.7	Who should read this document	2
2	The Problem and the Solution	3
2.1	The shift: from chat to action	3
2.2	What teams cannot answer today	3
2.3	Downstream effects	3
2.4	Why frameworks alone do not solve it	4
2.5	The Connector solution	4
2.5.1	What changes for the customer	4
2.6	What we sell first vs. what we expand into	4
3	How Connector Works (Semi-Technical)	6
3.1	The mental model	6
3.2	Two planes (simple)	6
3.3	What runs on the node	7
3.4	Request and evidence flow	7
3.5	Operator surfaces	7
3.6	Cage-internal DNS (why plugins feel like “apps”)	7
3.7	What “healthy node” means in a demo	8
4	Use Cases and Unique Selling Propositions	9
4.1	Use cases by industry segment	9
4.2	Use cases by buyer persona	10
4.3	USP table — why Connector wins the conversation	10
4.4	Competitive framing (simple)	11
4.5	Qualification checklist (3 questions)	11
5	Custom Workflows vs. Pre-Made Plugins	12
5.1	Definitions	12
5.2	Decision table	13
5.3	Why not only custom workflows?	13

5.4	Why not only pre-made plugins?	13
5.5	Typical adoption path	13
5.6	What to say in a meeting	14
6	Plugin Portfolio — All Nine Apps	15
6.1	Status legend	15
6.2	Master reference table	16
6.3	DevGuard Shipped	16
6.4	TraceTramp Shipped	16
6.5	WitnessCtl Shipped	17
6.6	LedgerLens Shipped	17
6.7	Conductor Planned	17
6.8	AgentLoop Planned	17
6.9	AgentPassport Planned	18
6.10	Relay Planned	18
6.11	Engram Planned	18
6.12	Progressive adoption narrative	18
7	Plugin Combinations, GTM Motion, and Talk Tracks	19
7.1	Three reference scenarios (from product marketing)	19
7.2	Minimum viable stacks by pain	19
7.3	GTM motion (2026)	19
7.3.1	Motion type	20
7.3.2	Four steps	20
7.3.3	Outreach framing that works	20
7.3.4	2026 targets (honest)	20
7.4	Pilot structure (reminder)	20
7.5	Why they pay (buying triggers)	21
7.6	Talk tracks — say vs. avoid	21
7.7	Pricing signal (directional)	21
7.8	Related documents	21
	Closing	22

Chapter 1

Executive Summary

1.1 What Connector is (one sentence)

Connector OS is a **self-hosted governance node** for AI agents and workflows: it sits between your agents and production systems and makes every consequential action **isolatable**, **policy-bound at runtime**, and **provable afterward**.

1.2 What we are not

- Not a chatbot or copilot product.
- Not “prompt tuning” or guardrails-only filtering.
- Not a replacement for your entire AI stack on day one.
- Not a giant transformation program sold as the first conversation.

1.3 The three pillars buyers remember

Isolate	Boundaries so agents, data, and tools do not blend blindly across teams or environments.
Govern	Policy, budgets, and tool access evaluated at runtime ; violations can be blocked, not only logged after the fact.
Verify	Tamper-evident receipts and exportable evidence—artifacts auditors and executives can review, not raw log dumps.

Operator idiom: *trace* → *explain* → *prove* (dashboard, API, and CLI).

1.4 Product shape: one node, many workloads

Customers install **one Connector OS node** (`connector-platform` + `connectorctl`). On that node they run:

- **First-party plugins** (DevGuard, TraceTramp, WitnessCtl, LedgerLens, and planned apps).

- **Custom workflows** (CLS / company-specific automation authored for their business).
- **Third-party AGOS plugins** (packaged .cpkg workloads from a hub model).

Think **operating system + applications**, not a bag of disconnected microservices.

1.5 Entry wedge (how we sell in 2026)

We enter through **one live workflow** where AI already takes action and mistakes hurt:

- support automation (refunds, routing, discounts),
- fintech ops (flags, reviews, exceptions),
- internal automation (approvals, case routing).

We sell **decision trace + explanation + compliance-ready output** for that workflow—not “the whole platform vision” in meeting one.

Core GTM line

When your AI makes a decision, Connector shows what happened, why it happened, and gives you evidence you can stand behind.

1.6 Stage

Controlled beta. Runtime, plugins, and workflows are exercised in self-hosted and lab settings with design partners. Position honestly: thesis is clear; distribution is founder-led; proof comes from narrow pilots.

1.7 Who should read this document

Reader	Use this for
GTM / sales	Pitch framing, use-case tables, plugin story, pilot scope
Semi-technical champion (VP Eng, Head of Platform)	How the node works, custom vs pre-made choice
Founder / investor conversation	Problem depth, USP, wedge, motion

Chapter 2

The Problem and the Solution

2.1 The shift: from chat to action

AI is no longer only drafting text. Teams deploy agents that:

- issue refunds and route support cases,
- approve or block transactions,
- update records and trigger downstream workflows,
- make recommendations that change customer experience or revenue.

When AI only generated copy, mistakes were annoying. When AI **acts**, mistakes become **operational**: revenue, trust, escalations, compliance, and internal confidence.

2.2 What teams cannot answer today

After an incident, most organizations struggle with three questions:

1. **What happened?**
2. **Why did it happen?**
3. **Can we prove it?**

Examples:

- Support cannot explain why the AI issued a refund.
- Fintech cannot reconstruct why a transaction was flagged.
- Internal automation teams cannot show why a workflow took a different path.

Human version of the pain

When something goes wrong, teams cannot clearly explain or prove what the AI did.

2.3 Downstream effects

- Automation slows down; humans pulled into firefighting.
- Buyers and customers lose trust.

- Security and compliance become blockers.
- AI initiatives lose momentum or stall in review.

2.4 Why frameworks alone do not solve it

Agent **frameworks** (orchestration libraries, LLM SDKs) help developers chain calls. They typically leave **governance, audit-grade evidence, cost enforcement, and recovery** to application code. In regulated or high-stakes environments that is insufficient:

- Log lines are not legal-grade evidence.
- Post-hoc guardrails do not stop a tool call already in flight.
- Token spend is invisible until the invoice arrives.
- Compliance cannot be credibly bolted on after production launch.

2.5 The Connector solution

Connector solves this at the **infrastructure boundary**—the place where agents commit state, spend money, call tools, and affect customers.

Solution in buyer language

Connector is the **execution control layer** for AI decisions: trace every step, enforce policy at runtime, export proof your stakeholders can review.

2.5.1 What changes for the customer

Before Connector	After Connector	Outcome
“The model decided that”	Full decision path with context	Faster incident review
Logs scattered across vendors	Signed receipts + exports	Audit-ready artifacts
Policy in prompts only	Runtime block / approve	Fewer silent failures
Surprise LLM bills	Per-agent / per-team attribution	Finance alignment
Shadow AI tools	Catalog of gov-erned workloads	Platform visibility

2.6 What we sell first vs. what we expand into

- **First sale:** one workflow, one pain, one pilot (2–6 weeks).
- **Expansion:** more workflows, more teams, more plugins on the same node.

- **Long-term:** Connector becomes how the company runs AI safely—not a single-feature tool.

Discipline for first calls

Do not lead with: “Here is our nine-plugin OS.”

Lead with: “Pick one workflow where AI makes decisions. We make it traceable, explainable, and provable.”

Chapter 3

How Connector Works (Semi-Technical)

This chapter explains the system without requiring a reader to open the architecture PDF.

3.1 The mental model

1. **Bring the node up** — install one tarball, run `connector-platform` (often via `connectorctl start`).
2. **Discover workloads** — dashboard and API list plugins, workflows, and health.
3. **Route traffic through governance** — agents and tools hit Connector boundaries (proxy, cage, witness layer).
4. **Evidence flows up** — traces, receipts, and exports bubble to operators and auditors.

3.2 Two planes (simple)

Hosted control plane (account)	Signup, billing, license keys, profile—customer portal.
Self-hosted node (runtime)	<code>connector-platform</code> on customer infra: API, embedded dashboard, plugins, workflows. Data stays on-prem unless they choose otherwise.

Key message for buyers: self-hosted runtime, optional hosted entitlement—no cloud lock-in for execution.

3.3 What runs on the node

Workload type	Examples	What it is
First-party plugins	DevGuard, TraceTramp, WitnessCtl, LedgerLens	Packaged capabilities maintained by Connector; install/enable on the node.
Custom workflows (CLS)	Support refund bot, internal approval flow	Customer-authored logic compiled/run under kernel governance.
AGOS / .cpkg plugins	Slack notifier, Jira bridge (ecosystem)	Third-party or reference plugins with manifest + handshake contract.

3.4 Request and evidence flow

Directionality

Requests flow down (operator or agent → policy → execution).

Evidence flows up (receipts, traces, cost → dashboard → export).

Typical path for a governed API call:

1. Agent or app sends request (directly or via TraceTramp proxy).
2. Connector evaluates policy, budget, and tool rules **before** side effects commit.
3. Allowed actions execute; blocked actions return a clear denial with reason.
4. WitnessCtl (and related stores) capture receipts aligned with trace metadata.
5. Operator exports HTML/PDF/JSON for review, SOC 2, or customer escalation.

3.5 Operator surfaces

- **Dashboard** (embedded in `connector-platform`) — Service map, plugins, compliance, agents.
- **connectorctl** — Bootstrap, health, plugin install, workflow lifecycle, hub commands.
- **HTTP API** — `/api/v1/*` for automation and integration.

3.6 Cage-internal DNS (why plugins feel like “apps”)

Enabled plugins can register under an internal TLD (default `.cnktros`) so operators reach management planes predictably, e.g. `tracetramp.cnktros`, without memorizing ad-hoc ports. Product story: **one node, discoverable apps**.

3.7 What “healthy node” means in a demo

1. `connectorctl doctor` or dashboard shows green health.
2. At least one plugin or workflow enabled.
3. A sample agent call produces a visible trace **and** an exportable receipt.
4. (Optional) A policy violation is blocked live—not replayed from logs.

That four-step demo is the minimum credible “how it works” for a semi-technical buyer.

Chapter 4

Use Cases and Unique Selling Propositions

4.1 Use cases by industry segment

Segment	Example workflow	Trigger event	What Connector proves
AI support / CX	Refund or discount automation	Wrong refund; escalation	Decision path + export for team lead
Fintech ops	Transaction review, fraud flags	Auditor or regulator question	Tamper-evident receipt chain
Internal automation	Approvals, routing, exceptions	Cross-team blame after failure	Trace + explain in plain language
Coding agents (eng)	IDE / CLI agent with shell access	Secret leak or destructive command	DevGuard block + live monitor
Platform ML teams	Multi-agent production fleet	No single pane of glass	TraceTramp graph + AgentLoop (planned) lifecycle
Finance FinOps	Uncontrolled model spend	Monthly bill shock	LedgerLens attribution + budget gates
Security GRC	Shadow AI, SOC 2 prep	Audit window opening	WitnessCtl exports + policy evidence
Healthcare (future wedge)	Clinical or PHI-adjacent agents	HIPAA review	Engram + WitnessCtl combination (planned)

Table 4.1: Primary use-case segments (2026 wedge focus: top three rows)

4.2 Use cases by buyer persona

Persona	Pain (their words)	Plugins / path	Outcome they buy
Developer	Framework lock-in; no safe memory	Relay + Engram (planned) or custom WF + DevGuard	Ship without rewriting stack
ML Engineer	Hallucinations; no grounding	Engram + WitnessCtl	Decisions with evidence
Platform Engineer	Fleet invisible; no standard egress	TraceTramp + AgentLoop (planned) + Conductor (planned)	One operational model
CTO	\$40K+ LLM burn; no breakdown	LedgerLens + TraceTramp	Cost under control
CISO	No audit trail; shadow AI	WitnessCtl + DevGuard + AgentPassport (planned)	Defensible AI posture
VP Support / CX	Cannot explain AI re-funds	WitnessCtl + custom support WF	Faster incidents, customer trust
Head of Risk (fin-tech)	Unexplained model flags	TraceTramp + WitnessCtl + LedgerLens	Regulatory defensibility

Table 4.2: Persona-to-outcome map (for discovery calls)

4.3 USP table — why Connector wins the conversation

USP	Alternatives fail because...	Connector difference
Runtime enforcement	Guardrails filter text in/out; observability tools log after the fact	Policy and budgets at the action boundary
Proof, not logs	SIEM lines lack decision semantics	Cryptographic receipts + structured exports (HTML/PDF/JSON)
Self-hosted node	Cloud AI safety is single-vendor; data leaves estate	One binary on customer infra; hosted portal only for account/license
Progressive adoption	Rip-and-replace platforms scare procurement	Start with one plugin + one workflow; expand on same node
OS mental model	Point tools solve one slice (cost OR safety OR trace)	Kernel + plugins: combine DevGuard + TT + WitnessCtl as needed
Trace → explain → prove	Buyers hear “AI governance” as vague	Concrete operator idiom matching how SREs work
Pilot-shaped GTM	Enterprise platforms need 6-month POVs	2–6 week single-workflow pilot with documented win

Table 4.3: Unique selling propositions vs. common alternatives

4.4 Competitive framing (simple)

Category	Why we still win
LLM safety proxies	Input/output only; no tool execution governance or proof chain
AI observability (LangSmith, etc.)	Evaluation and tracing; not runtime block + cryptographic evidence
Cloud guardrails	Single cloud, model layer; not cross-stack agent OS
Agent frameworks	Composition; not kernel-level isolation, budgets, receipts

4.5 Qualification checklist (3 questions)

1. Is your AI already **taking actions**, not only producing text?
2. When it makes the wrong call, can you **explain why**?
3. Have you had an incident—or do you **expect one soon**?

If yes / no / yes → prioritize for outreach.

Chapter 5

Custom Workflows vs. Pre-Made Plugins

This is one of the most common semi-technical questions: *“Should we build our own workflow or turn on WitnessCtl / TraceTramp / DevGuard?”*

5.1 Definitions

- **Pre-made (first-party) plugins** — Maintained Connector applications (DevGuard, TraceTramp, WitnessCtl, LedgerLens, and planned Conductor, AgentLoop, etc.). Installed on the node; solve a **horizontal** problem (safety, trace, audit, cost).
- **Custom workflows (CLS)** — Customer-specific automation: your refund logic, your risk rules, your internal routing. Authored for **your** business process; still runs **on** the Connector kernel so governance applies.
- **Community / AGOS plugins** — Packaged `.cpkg` integrations (Slack, Jira, Datadog patterns). Faster than bespoke code for standard connectors.

5.2 Decision table

Choose custom workflow when...	Choose pre-made plugin when...	Notes
The logic is unique to your business	The need is industry-standard (audit, proxy, finops, coding safety)	Plugins are faster time-to-value
You already have agent code you cannot rewrite	You need a control point before production traffic	TraceTramp sits in the water line
The buyer cares about <i>this</i> refund policy	The buyer cares about <i>any</i> governed API call	WitnessCtl is horizontal
You are proving ROI on one operational flow	You are proving security/-compliance posture	Pilot often = custom WF + 1-2 plugins
Engineering owns the business rules	Platform/security owns the boundary	Different champions, same node

5.3 Why not only custom workflows?

Building everything yourself recreates what the plugins already productize:

- A proxy with metering, quarantine, and policy versioning (TraceTramp).
- Tamper-evident session export and compliance shells (WitnessCtl).
- Host-level enforcement for coding agents (DevGuard).
- CFO-grade attribution and budget gates (LedgerLens).

GTM rule: sell custom workflow **for the business story**; sell plugins **for the control plane**. Almost every serious deployment uses **both**.

5.4 Why not only pre-made plugins?

Plugins do not know your company's refund rules, credit policy, or internal escalation matrix. Without a custom workflow (or heavy configuration), you only get **infrastructure**, not **your** automated decision.

Combined pitch

Pre-made plugins answer: *"How do we govern AI in production?"*

Custom workflows answer: *"How do we automate **our** decision safely?"*

5.5 Typical adoption path

1. **Week 1-2:** Node up + one plugin matching acute pain (often WitnessCtl or TraceTramp).

2. **Week 2–4:** Wire **one** custom workflow through that boundary.
3. **Month 2+:** Add plugins (LedgerLens for cost, DevGuard for eng agents).
4. **Quarter 2+:** More workflows on catalog; platform team standardizes patterns.

5.6 What to say in a meeting

“We are not asking you to throw away your agent. We give you a governed runtime boundary—TraceTramp or WitnessCtl—and your workflow runs inside it. You keep your business logic; we make it provable.”

Chapter 6

Plugin Portfolio — All Nine Apps

Connector markets **nine plugins** orbiting one kernel. **Four** have substantial in-repo implementation today; **five** are on the product roadmap with UI stubs and documented positioning.

6.1 Status legend

- **Shipped** — In repository, usable in lab/beta deployments.
- **Beta** — Partial UI or active development.
- **Planned** — Positioned on website/roadmap; not yet a full production app.

6.2 Master reference table

Plugin	Status	One-line role	When to lead with it
DevGuard	Shipped	Policy enforcement for files, commands, secrets in coding agents	Buyer has Cursor/Copilot/CLI agents; eng security story
TraceTramp	Shipped	Runtime proxy: meter, filter, quarantine, full execution graph	Visibility + enforcement on API traffic without re-instrumenting apps
WitnessCtl	Shipped	API witness: capture, seal, export audit receipts	SOC 2, audits, “prove what the AI did”
LedgerLens	Shipped	AI FinOps: attribution, anomalies, budget gates, CFO export	CFO/CTO pain on LLM spend
Conductor	Planned	Multi-agent orchestration without a framework; declarative pipelines	Multiple agents must coordinate with runtime enforcement
AgentLoop	Planned	Lifecycle: spawn, monitor, suspend, recover at scale	Large agent fleet; platform engineering buyer
AgentPassport	Planned	W3C DID identity per agent: verifiable, portable, revocable	Identity / zero-trust / cross-org agent trust
Relay	Planned	Register any HTTP endpoint; get Connector governance without framework swap	Fastest “first governed call” demos
Engram	Planned	Enterprise memory: entropy-scored, dehallucination, audit-chained	Grounding, memory safety, regulated knowledge

Table 6.1: Nine-plugin portfolio (status as of 2026 beta)

6.3 DevGuard Shipped

Tagline: Policy enforcement for every file, command, and secret your agent touches.

- **Problem:** Coding agents can run shell commands, read secrets, and modify repos without a single enforcement point.
- **What it does:** Cage mode, git hooks, exec hardening, live dashboard, approval flows for blocked actions.
- **Buyer:** VP Engineering, security-minded CTO, platform team supporting dev tools.
- **Not a substitute for:** API-level witness (pair with WitnessCtl for full audit story).

6.4 TraceTramp Shipped

Tagline: Full execution graph for every agent call—language agnostic, proxy at the ingress.

- **Problem:** Traffic bypasses observability; policies are inconsistent across services.

- **What it does:** Data plane proxy (default **Control** pipeline = enforce + observe), management plane for tenants/policies/budgets, evidence plane for exports.
- **Buyer:** Platform engineer, ML ops lead.
- **Pairs with:** WitnessCtl (handoff for SIEM / audit alignment).

6.5 *WitnessCtl* Shipped

Tagline: Cryptographic audit receipts for every agent action—tamper-evident, export-ready.

- **Problem:** Logs are not defensible; auditors want artifacts.
- **What it does:** Session capture, decision digest (firewall, kernel snapshot, TraceTramp hints), export JSON/CSV/Markdown/HTML/PDF.
- **Buyer:** CISO, compliance lead, risk officer.
- **Demo moment:** Export HTML → Print to PDF for auditor walkthrough.

6.6 *LedgerLens* Shipped

Tagline: Real-time cost attribution per agent, team, and model—with hard budget gates.

- **Problem:** “We spent \$47K on GPT-4” with no owner or lever.
- **What it does:** Executive dashboard, chargeback pivots, anomaly alerts, budget policies (alert, downgrade, hard-stop, queue), CFO export package.
- **Buyer:** CTO, FinOps, CFO office.
- **Pairs with:** TraceTramp (traffic metering) for accurate attribution.

6.7 *Conductor* Planned

Tagline: Multi-agent orchestration without adopting another framework.

- **Problem:** Hand-rolled orchestration lacks runtime policy and recovery.
- **Future value:** Declarative pipelines with kernel-enforced caps—OS-level orchestration, not just DAG scripts.
- **Buyer:** Platform team running 10+ cooperating agents.

6.8 *AgentLoop* Planned

Tagline: Agent lifecycle management—spawn, monitor, suspend, recover.

- **Problem:** No standard lifecycle for long-running agents at scale.
- **Future value:** DNS-style agent addressing, mesh routing, workers, observability tiers (per product docs).
- **Buyer:** Infrastructure lead treating agents like microservices.

6.9 AgentPassport Planned

Tagline: W3C DID identity for every agent.

- **Problem:** Agents share API keys; no portable identity or revocation story.
- **Future value:** Verifiable agent identity across systems and partners.
- **Buyer:** Security architect, B2B platforms issuing agent credentials.

6.10 Relay Planned

Tagline: Zero-framework runtime—register an HTTP endpoint, get governance.

- **Problem:** Teams want governance without rewriting to a new SDK.
- **Future value:** Fastest path: point base URL at Relay, see audit log, add budget lines in config.
- **Buyer:** Developer who needs a win in one sprint.

6.11 Engram Planned

Tagline: Enterprise memory for agents—entropy-scored, dehallucination enforced.

- **Problem:** Ad-hoc RAG and chat memory are not governed or auditable.
- **Future value:** Memory writes chained to audit; quality scoring for grounding.
- **Buyer:** ML engineer, healthcare/regulated knowledge use cases.

6.12 Progressive adoption narrative

“Use one plugin or all nine. Start where it hurts—WitnessCtl for audit, LedgerLens for spend, DevGuard for coding agents—then add workflows on the same node.”

Do not promise all nine as GA in the first meeting unless the roadmap has been confirmed for that account.

Chapter 7

Plugin Combinations, GTM Motion, and Talk Tracks

7.1 Three reference scenarios (from product marketing)

Scenario	Plugin stack	Story
Support AI	Relay + Engram + WitnessCtl (planned mix)	Governed support agent with memory + export for escalations
Fintech compliance agent	DevGuard + LedgerLens + WitnessCtl + AgentPassport	Tool safety, spend caps, audit, identity
Multi-agent re-search	Conductor + AgentLoop + Engram + TraceTramp	Coordinated agents with trace and memory

Table 7.1: Reference plugin combinations by scenario

For **2026 beta pilots**, recommend stacks that are mostly **Shipped**: e.g. **TraceTramp** + **WitnessCtl** + **one custom workflow**, or **DevGuard** + **WitnessCtl** for engineering-led accounts.

7.2 Minimum viable stacks by pain

Acute pain	Start here
“Cannot explain AI decisions”	WitnessCtl + custom workflow
“Agents bypass our policies”	TraceTramp (Control pipeline) + WitnessCtl
“Coding agent almost leaked secrets”	DevGuard (+ WitnessCtl for exports)
“LLM bill out of control”	LedgerLens + TraceTramp
“Need to ship one governed agent fast”	Relay (when GA) or TraceTramp proxy path today

7.3 GTM motion (2026)

7.3.1 Motion type

- **Founder-led** direct outreach.
- **Network-based** warm intros.
- **Not** broad paid acquisition or “platform tour” webinars yet.

7.3.2 Four steps

1. **Identify** teams with live AI actions + recent or expected incident.
2. **Sell one workflow** — one pain, one pilot (2–6 weeks).
3. **Land evidence** — customer quote, export sample, before/after trace.
4. **Expand** — more workflows, more plugins, reference story.

7.3.3 Outreach framing that works

Opening email / LinkedIn DM

Are you using AI to take actions in support or ops?
When it makes the wrong call, can you explain why?
If not, we should talk.

7.3.4 2026 targets (honest)

By end of Q2:

- 3–5 qualified conversations in wedge segments.
- 1–2 paid pilots in progress or completed.
- At least one documented pilot win with customer statement.

By end of Q3:

- 2–3 completed pilots with outcomes.
- At least 1 expansion conversation (more workflows or teams).
- Messaging refined from real objections.

7.4 Pilot structure (reminder)

- **Scope:** one workflow, one team, defined success criteria.
- **Deliverables:** full trace, decision explanation, audit-ready export.
- **Success reaction:** “Now we can see what happened.”
- **Not:** free indefinite POC of entire platform.

7.5 Why they pay (buying triggers)

- Unexplained AI decisions slowing operations.
- Customer escalations they cannot defend.
- Deals blocked by prospect security questions.
- Compliance review approaching without artifacts.
- Internal teams afraid to ship more AI without control.

7.6 Talk tracks — say vs. avoid

Say	Avoid
Execution control for AI decisions	“Revolutionary AI platform”
Trace, explain, prove	Leading with MemPacket, CNP, or CLS internals
One workflow pilot in 2–4 weeks	“Rip and replace your stack”
Self-hosted node, your data	Overclaiming GA for all nine plugins
Isolate, govern, verify	Competing on model quality

7.7 Pricing signal (directional)

Tier	Signal	Value
Pilot	Time-boxed, scoped	One workflow, proof bundle, co-produced evidence
Startup	Monthly node fee	Bounded agents, core plugins, exports
Scale	Higher node / SLA	Multi-team, full plugin set, operations support
Enterprise	Custom	Air-gap, dedicated node, custom compliance

Pricing follows **infrastructure** logic (per node / per agent), not per seat.

7.8 Related documents

- [platform/docs/arch/latex/CONNECTOR_SYSTEM_ARCHITECTURE.pdf](#) — full technical depth.
- [platform/docs/cofounder_clarity_latex/CONNECTOR_COFOUNDER_CLARITY_2026.pdf](#) — wedge, pilot, cofounder alignment.
- [platform/docs/gtm/connector_investor_one_pager.pdf](#) — short investor brief.
- [platform/docs/gtm/design-partner-targets-practical-v2.pdf](#) — outreach target list.

Closing

Connector wins when buyers feel an **expensive, present risk**—unexplained AI actions—and we make that risk **visible, bounded, and provable** on their infrastructure.

Start narrow. Prove one workflow. Expand on one node.